

OS LAB 9:

Questions

1. First, let's make sure you understand how the programs generally work, and some of the key options. Study the code in vector-deadlock.c, as well as in main-common.c and related files. Now, run `./vector-deadlock -n 2 -l 1 -v`, which instantiates two threads (-n 2), each of which does one vector add (-l 1), and does so in verbose mode (-v). Make sure you understand the output. How does the output change from run to run?
2. Now add the -d flag, and change the number of loops(-l)from 1 to higher numbers. What happens? Does the code (always) deadlock?
3. How does changing the number of threads (-n) change the outcome of the program? Are there any values of -n that ensure no deadlock occurs?
4. Now examine the code in vector-global-order.c. First, make sure you understand what the code is trying to do; do you understand why the code avoids deadlock? Also, why is there a special case in this vector_add() routine when the source and destination vectors are the same?
5. Now run the code with the following flags: `-t -n 2 -l 100000 -d`. How long does the code take to complete? How does the total time change when you increase the number of loops or the number of threads?

1.

```
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 1 -v
->add(0, 1)
->add(0, 1)
<-add(0, 1)
<-add(0, 1)
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 1 -v
->add(0, 1)
->add(0, 1)
<-add(0, 1)
<-add(0, 1)
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 1 -v
->add(0, 1)
->add(0, 1)
<-add(0, 1)
<-add(0, 1)
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 1 -v
->add(0, 1)
->add(0, 1)
<-add(0, 1)
<-add(0, 1)
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 1 -v
->add(0, 1)
->add(0, 1)
<-add(0, 1)
<-add(0, 1)
```

The program creates two threads, and each thread performs one vector addition operation. When running with the -v (verbose) option, the output shows when a thread starts (->add(x, y)) and finishes (<-add(x, y)) a vector addition between two vectors.

The output changes from run to run because thread scheduling is non-deterministic. Depending on how the operating system schedules the threads, their executions may interleave differently. In some runs, one thread completes its entire vector addition before the other thread starts, resulting in a clean pair of start and finish messages. In other runs, both threads may start before either finishes, causing the start messages to appear first and the finish messages to appear later in a mixed order.

This variation demonstrates concurrent execution and illustrates that the order of thread execution is not deterministic, even when the program and inputs remain the same.

2.

```
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 2 -v -d
->add(1, 0)
->add(0, 1)
<-add(1, 0)
<-add(0, 1)
->add(0, 1)
->add(1, 0)
<-add(0, 1)
<-add(1, 0)
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 2 -v -d
->add(1, 0)
->add(0, 1)
<-add(1, 0)
<-add(0, 1)
->add(0, 1)
->add(1, 0)
<-add(0, 1)
<-add(1, 0)
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-deadlock -n 2 -l 2 -v -d
->add(1, 0)
->add(0, 1)
<-add(0, 1)
<-add(1, 0)
->add(1, 0)
->add(0, 1)
<-add(1, 0)
<-add(0, 1)
```

I also got the ./vector-deadlock -n 2 -l 100 -v -d multiple times

[illegible]

The code does not deadlock.

Reason:

When the -d flag is added, the program deliberately introduces a deadlock-prone locking order when accessing the vectors. As the number of loops (-l) is increased, the threads perform more vector addition operations, which increases the chance that two threads will each acquire one lock and then wait indefinitely for the other lock.

As a result, the program may appear to hang, producing no further output. This indicates that a deadlock has occurred.

However, the code does **not always** deadlock. Deadlock depends on the timing and scheduling of threads. In some runs, the threads may complete all iterations without deadlocking; in other runs, they may deadlock early or

after many iterations. The behavior is therefore non-deterministic and becomes more likely as $-l$ increases, but it is not guaranteed in every execution.

3.

[illegible]

Changing the number of threads (-n) directly affects the likelihood of deadlock. As the number of threads increases, more threads compete to acquire the same locks, which increases contention and makes deadlock more likely.

With only one thread (-n 1), deadlock cannot occur because a single thread can never wait on a lock held by another thread. Therefore, -n = 1 guarantees no deadlock.

With two or more threads ($n \geq 2$), deadlock becomes possible when threads acquire locks in different orders. As n grows larger, the probability of deadlock generally increases, but deadlock is still not guaranteed because thread scheduling is non-deterministic.

In summary, only $-n = 1$ ensures that no deadlock occurs; any value greater than 1 can potentially result in deadlock.

4.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#include "mythreads.h"

#include "main-header.h"
#include "vector-header.h"

void vector_add(vector_t *v_dst, vector_t *v_src) {
    if (v_dst < v_src) {
        Pthread_mutex_lock(&v_dst->lock);
        Pthread_mutex_lock(&v_src->lock);
    } else if (v_dst > v_src) {
        Pthread_mutex_lock(&v_src->lock);
        Pthread_mutex_lock(&v_dst->lock);
    } else {
        // special case: src and dst are the same
        Pthread_mutex_lock(&v_src->lock);
    }
    int i;
    for (i = 0; i < VECTOR_SIZE; i++) {
        v_dst->values[i] = v_dst->values[i] + v_src->values[i];
    }
    Pthread_mutex_unlock(&v_src->lock);
    if (v_dst != v_src)
        Pthread_mutex_unlock(&v_dst->lock);
}

void fini() {}

#include "main-common.c"
```

The `vector_add()` function avoids deadlock by enforcing a global ordering on lock acquisition. The order is determined by comparing the memory addresses of the source and destination vectors

- If $v_dst < v_src$, the destination vector's lock is acquired first, followed by the source vector's lock.

- If $v_dst > v_src$, the source vector's lock is acquired first, followed by the destination vector's lock.

Because all threads use the same comparison rule, they will always acquire locks in the same order for any given pair of vectors. This consistent ordering prevents circular wait, which is a necessary condition for deadlock. As a result, deadlock cannot occur in this design.

Reason for the special case ($v_dst == v_src$)

When the source and destination vectors are the same ($v_dst == v_src$), locking both vectors would mean attempting to lock the same mutex twice. Since POSIX mutexes are non-recursive by default, this would cause the thread to block on itself, creating a self-deadlock.

To avoid this, the code includes a special case where:

- The mutex is locked only once
- The vector update is performed safely
- The mutex is unlocked once at the end

This ensures correctness while avoiding unnecessary locking and self-deadlock.

5.

```
hitori@hitori:~/OS_Lab_9_10_code$ ./vector-global-order -t -n 2 -l 100000 -d  
Time: 0.11 seconds
```

It takes **0.11** seconds to run the code completely.

Effect of increasing the number of loops (-l)

As the number of loops increases, the total execution time increases approximately linearly. This is because each loop performs the same amount of work (locking mutexes and adding vector elements). Doubling -l roughly doubles the runtime.

Effect of increasing the number of threads (-n)

Increasing the number of threads does not always improve performance. In fact:

- With more threads, there is more lock contention on the vector mutexes.
- Threads often block while waiting for locks, reducing parallelism.
- As a result, execution time may increase or scale poorly as $-n$ grows.

In many cases, adding more threads beyond a small number actually makes the program slower, not faster.